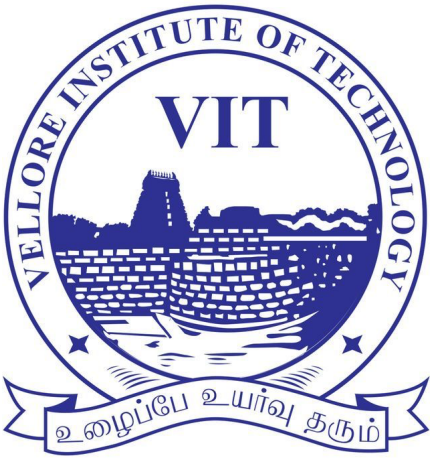
VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BCSE307P – COMPILER DESIGN LAB

PROJECT REPORT



CodeGuard: An Online Code Execution Platform with Logic-Level Plagiarism Detection

Submitted by

Gauransh Purohit – 24BKT0166

Aayush Purohit – 23BCE0378

Anika Shishodia – 24BCI0120

Under the guidance of

Nithya K,
Assistant Professor Senior Grade II

SEPTEMBER, 2026

BONAFIDE CERTIFICATE

This is to certify that the project work titled CodeGuard: An Online Code Execution Platform with Logic-Level Plagiarism Detection is a bonafide record of work carried out by Gauransh Purohit (24BKT0166), Aayush Purohit (23BCE0378) and Anika Shishodia (24BCI0120), submitted in partial fulfilment of the requirements for the course BCSE307P – Compiler Design Lab, School of Computer Science and Engineering, VIT Vellore, during the academic year 2026 - 2027.

Faculty Guide: NITHYA K

Date: 14 September 2026

DECLARATION

We hereby declare that the project report titled CodeGuard: An Online Code Execution Platform with Logic-Level Plagiarism Detection submitted by us to VIT Vellore in partial fulfilment of the requirement for the award of the degree/course BCSE307P – Compiler Design Lab is a record of bona fide work carried out by us under the supervision of Nithya K, Assistant Professor Senior Grade II. The contents of this report, in full or in part, have not been submitted to any other institute or university for the award of any degree or diploma.

Place: VIT Vellore

Date: 14 September 2026

Gauransh Purohit - 24BKT0166

Aayush Purohit - 23BCE0378

Anika Shishodia - 24BCI0120

ABSTRACT

Online coding platforms and programming laboratories let students compile and run code, but they rarely verify originality, and conventional similarity checkers are defeated by simple edits such as renaming variables, reformatting code, or rewriting a for loop as a while loop. This project presents CodeGuard, a web-based platform that combines a code editor and sandbox with a two-level plagiarism detector built from compiler-construction techniques. Hand-written lexers convert source code into a normalised token stream in which identifiers and literals are abstracted; k-gram hashing with the winnowing algorithm produces fingerprints that give a token-level similarity score. Recursive-descent parsers build abstract syntax trees (AST), a canonicalisation pass rewrites equivalent constructs into one form, and recursive tree alignment with Merkle-style subtree hashing gives a logic-level score. Front-ends for C, C++, Python and Java allow submissions in different languages to be compared, and a submission database supports comparison against stored work. In a demonstration, a C factorial program compared with a Python factorial of the same algorithm scored 54% at token level but 95% at logic level, which shows why both levels are needed. Early prototype tests also showed that comparing unnormalised ASTs ranks restructured code incorrectly, which motivated the canonicalisation pass.

TABLE OF CONTENTS

CHAPTER 1 6
INTRODUCTION 6
1.1 Background 6
1.2 Problem Statement 6
1.3 Objectives & Novelty 6
CHAPTER 2 8
LITERATURE SURVEY 8
2.1 Existing Systems / Related Work 8
2.2 Research Gap 8
CHAPTER 3 9
SYSTEM DESIGN 9
3.1 Proposed System Architecture 9
3.2 Module Description 10
3.3 Compiler Phases Implementation 10
CHAPTER 4 12
IMPLEMENTATION DETAILS 12
4.1 Screenshots 12
4.2 Current Progress and Challenges 17
4.3 Conclusion and Future Enhancements 17
REFERENCES 19

CHAPTER 1

INTRODUCTION

1.1 Background

Programming is taught and assessed through laboratory exercises, online judges and coding contests, and the volume of submissions has grown far beyond what an instructor can read by hand. Platforms such as online judges can compile and run a submission and check its output, but a correct output says nothing about whether the student wrote the program. Copying from a classmate or from the internet is easy, and it is just as easy to disguise: renaming variables, changing indentation, adding comments, reordering independent statements, or turning a for loop into an equivalent while loop.

Compiler-construction theory offers a principled way to see through such disguises. A lexer discards formatting and comments and can abstract names; a parser exposes the grammatical structure of a program as an abstract syntax tree (AST); and normalisation passes can rewrite equivalent constructs into one canonical form. Comparing programs after these phases compares what the code does rather than how it looks. This project applies those front-end phases to build a platform that runs code and measures how much of it resembles other submissions, at both the token level and the logic level.

1.2 Problem Statement

Existing plagiarism checkers fall short in three ways. First, plain text or line-based comparison is defeated by cosmetic edits. Second, token-based tools handle renaming well but have no notion of program structure, so restructured code (for example a for loop rewritten as a while loop, or independent statements reordered) looks different even though the logic is the same. Third, most tools work on one programming language at a time and are separate from the system that compiles and runs the code, so a Python submission cannot be compared with a C submission of the same algorithm.

The problem addressed here is to design and build an integrated system that (i) compiles and executes submitted C/C++ code safely, (ii) reports a token-level similarity percentage for surface copying and a logic-level similarity percentage for structurally disguised copying, and (iii) extends both analyses to other languages by converting Python and Java submissions into C through a purpose-built converter.

1.3 Objectives & Novelty

Objectives

- To build a hand-written lexer for C/C++ that produces a normalised token stream (identifiers, numbers and strings abstracted) suitable for similarity analysis.
- To detect surface-level copying by k-gram fingerprinting with the winnowing algorithm and to report Jaccard and containment similarity.

- To build a recursive-descent parser that constructs an AST for a C subset, with error recovery so that a partial tree is produced for imperfect student code.
- To canonicalise equivalent constructs (for/while loops, redundant blocks) and to compare ASTs by tree alignment and subtree hashing in order to detect logic-level plagiarism.
- To compile and execute submissions in a resource-limited sandbox and report compiler diagnostics and program output.
- To convert typed subsets of Python and Java into C through a shared intermediate representation, so that submissions in these languages can be executed and compared with the same engine.

Novelty

- The individual techniques (winnowing, tree comparison, source-to-source translation) are established. The novelty lies in their combination: one pipeline that translates a submission into compilable C, scores it at token and logic level, and runs it in a sandbox. In our literature and patent search we did not find a system that does all three.
- The structural comparison is preceded by a canonicalisation pass, so that logically equivalent rewrites do not reduce the similarity score.
- The core compiler phases are written from scratch rather than taken from a parser generator, so every phase can be inspected and demonstrated.

CHAPTER 2

LITERATURE SURVEY

2.1 Existing Systems / Related Work

Source-code plagiarism detection has been studied for decades, and several families of work are relevant to this project. Table 2.1 summarises the systems, patents and papers we reviewed.

Table 2.1: Related work reviewed

Work	Technique	Limitation relative to this project
MOSS / Winnowing (Schleimer et al., 2003)	k-gram hashing; selects the local minimum hash in each window to form a document fingerprint	Token-level only; structural rewrites are not recognised; no execution or translation
JPlag (Prechelt et al., 2002)	Language-specific token abstraction followed by greedy string tiling	Each language is analysed in its own track; no translation to a common executable language
Zeidman patent family (e.g. US 7,503,035 B2)	Array-based comparison of source files; compares code against commented-out code	Single language; no AST layer; no execution environment
DECKARD (Jiang et al., 2007)	Characteristic vectors of parse-tree subtrees grouped by locality-sensitive hashing	Clone detector within one language; no canonicalisation of loop forms; no execution
US 11,099,842	AST converted to a control-flow representation to build a digital fingerprint	Single language; no cross-language translation; no judging component
Online judge with dynamic plagiarism detection (Iffath et al., 2021)	Sandboxed judge combined with winnowing-based fingerprint comparison	Token level only; single submission language
TransCoder (Roziere et al., 2020)	Unsupervised neural translation between C++, Java and Python	Translation only; output correctness is not guaranteed; no plagiarism analysis
US 8,869,120 and US 8,640,107	Source from several languages mapped to a common representation for static analysis	Targets rule checking, not plagiarism; does not emit compilable target-language code

Beyond these, tree-based comparison rests on the ordered tree edit distance of Zhang and Shasha, and token clone detection on the work of Kamiya et al. (CCFinder). Our tree-alignment score is a simplified, dynamic-programming relative of tree edit distance that is tractable for assignment-sized programs.

2.2 Research Gap

Reviewing the work above, three gaps remain that this project addresses.

- **No structural awareness in token tools.** Winnowing-style tools are robust to renaming but not to restructuring such as for-to-while rewrites. A canonicalised AST comparison closes this gap.
- **Language silos.** Detection is performed inside one language. A converter that emits compilable C lets a Python or Java submission be compared with a C submission of the same algorithm.
- **Separation of judging and plagiarism checking.** Judging systems and similarity checkers are usually separate. Here the same generated C code feeds both the sandbox and the similarity engine, so one submission gives correctness, a token score and a logic score.

CHAPTER 3

SYSTEM DESIGN

3.1 Proposed System Architecture

Figure 3.1 shows the overall workflow. A submission enters the system in C, C++, Python or Java. Python and Java sources pass through a language front-end (lexer and parser), are mapped onto a shared intermediate representation (IR), and are emitted as C by the code generator. C and C++ sources skip this step. The resulting C/C++ source then feeds three independent paths: a token path (lexer and winnowing) that yields the token-level score, a structure path (parser, canonicaliser, tree alignment and subtree hashing) that yields the logic-level score, and an execution path in which the code is compiled and run in a sandbox.

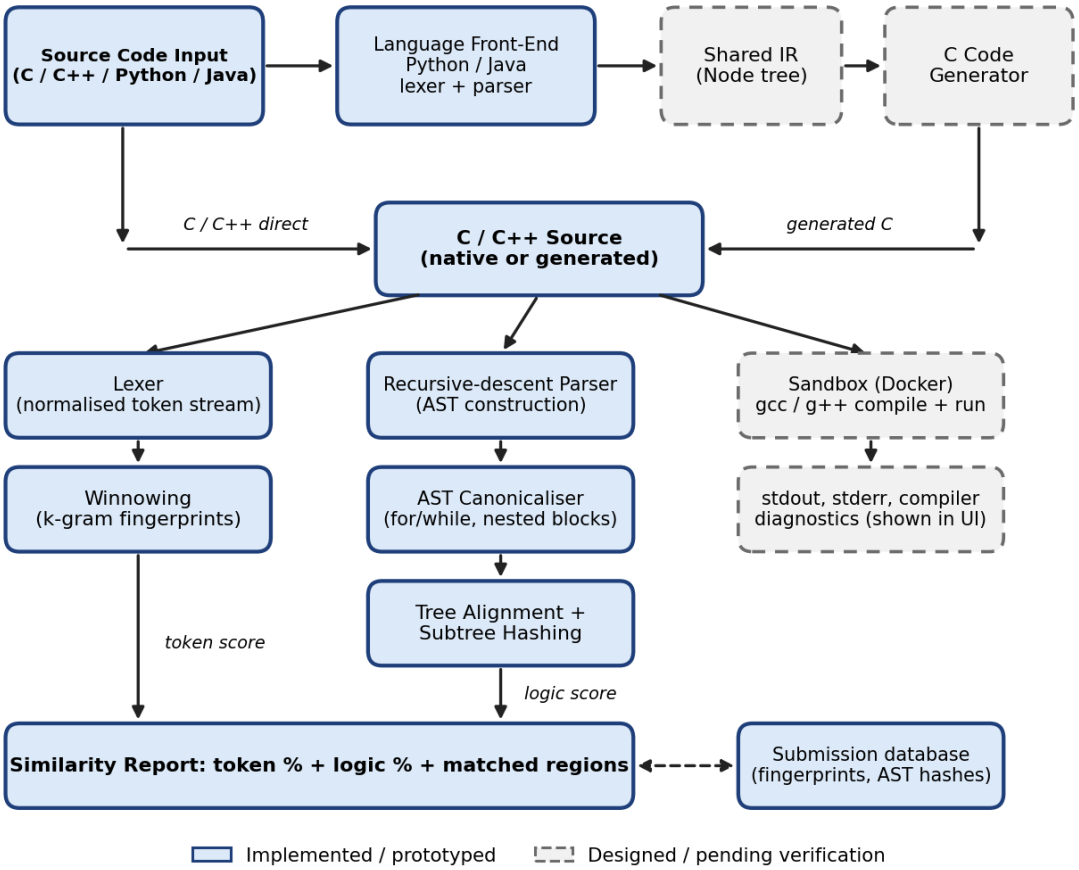


Figure 3.1: *Block diagram of the CodeGuard compiler workflow and its components*

The deployment follows three tiers: a React front-end with a code editor and report views; a FastAPI back-end that hosts the conversion, execution and plagiarism services; and a database that stores submissions, fingerprints and AST hashes so that a new submission can be compared with earlier ones without re-parsing them.

3.2 Module Description

Table 3.1 lists the modules of the system and the technique each one uses.

Table 3.1: Modules of CodeGuard

Module	Description
Lexer	Hand-written character scanner for C/C++. Emits keyword, identifier, number, string, character, operator, punctuation and preprocessor tokens with line and column. Identifiers become ID, numbers NUM, strings STR and characters CHAR; comments and whitespace are discarded.
Winnowing engine	Rolling polynomial hash over k-grams of the normalised token stream (k = 5, window w = 4); keeps the minimum hash per window. Produces Jaccard and containment similarity and matched positions.
Parser and AST	Recursive-descent parser for a C subset with twelve expression precedence levels and synchronisation-point error recovery. Builds a Node tree (kind, value, children, line).
AST canonicaliser	Rewrites for loops and while loops into one Loop node, hoists for-initialisers, and flattens redundant nested blocks, so equivalent constructs have the same tree shape.
AST similarity engine	Order-preserving tree alignment by dynamic programming (Dice-normalised) and Merkle-style subtree hashing that reports the maximal matching subtrees; the logic score is the larger of the two signals.
Language front-ends (Python, Java)	Lexer and parser for a typed Python subset (indentation handled with INDENT/DEDENT tokens) and a Java subset, each producing the shared IR.
C code generator (planned)	Pretty-prints the shared IR as readable, compilable C, adding only the headers that are needed.
Execution sandbox (planned)	Compiles with gcc/g++ and runs inside a Docker container with no network, CPU, memory and time limits and an output cap; returns stdout, stderr, diagnostics and exit status.
Web API and interface	FastAPI endpoints for convert, run, plagiarism check and submission history; React editor with report and pipeline-inspection views.
Submission database	Stores submissions, token fingerprints and subtree hashes; an inverted index shortlists candidates before the costlier alignment step.

3.3 Compiler Phases Implementation

Table 3.2 maps the classical phases of a compiler to what has been implemented in this project.

Table 3.2: Compiler phases in CodeGuard

Phase	Implementation in CodeGuard	Status
Lexical analysis	Hand-written scanner with longest-match operators, comment removal and token normalisation	Implemented
Syntax analysis	Recursive-descent parser with precedence climbing and error recovery	Implemented (C subset)
Semantic normalisation	Canonicalisation of equivalent constructs; identifier abstraction in place of a symbol table	Implemented; evaluation pending
Intermediate representation	Shared Node tree used for C and, later, Python and Java	Implemented for C
Analysis passes	Winnowing fingerprints, tree alignment, subtree hashing	Implemented
Code generation	IR to C source text; Python and Java front-ends	Designed

Lexical analysis. The scanner reads the source one character at a time. It recognises identifiers and keywords, decimal, hexadecimal and floating-point numbers with suffixes, string and character literals with escape sequences, and operators using longest-match (for example >>= before >> before >). Line and block comments are skipped, and an unterminated block comment raises an error with its line and column. A preprocessor line, including backslash continuations, becomes a single token. Characters outside the

language are skipped so that one stray symbol does not stop the analysis. After scanning, identifiers and literals are replaced by placeholders, which makes the token stream independent of naming.

Syntax analysis. The parser follows the grammar below, written in EBNF, with one function per non-terminal. Binary operators are handled by precedence climbing across twelve levels, from assignment down to multiplication, followed by unary, postfix and primary expressions. If a statement cannot be parsed, the parser skips to the next semicolon or closing brace and continues, so that a partial AST is still available. Constructs outside the subset (for example classes and templates) are skipped at top level rather than aborting.

```
program       ::= { function_decl | declaration }
function_decl ::= type IDENT "(" [ params ] ")" ( block | ";" )
block         ::= "{" { statement } "}"
statement     ::= decl_stmt | if_stmt | while_stmt | for_stmt
                | return_stmt | "break" ";" | "continue" ";"
                | block | expr_stmt
if_stmt       ::= "if" "(" expr ")" statement [ "else" statement ]
while_stmt    ::= "while" "(" expr ")" statement
for_stmt      ::= "for" "(" ( decl_stmt | expr_stmt ) [ expr ] ";" [ expr ] ")" statement
expr          ::= assignment { "," assignment }
assignment    ::= logical_or [ assign_op assignment ]
```

Figure 3.2: *Grammar of the supported C subset (excerpt)*

Semantic normalisation. Full type checking of C is left to gcc, which runs in the sandbox. The project instead performs a normalisation pass that serves the comparison. A for loop becomes its initialiser followed by a Loop node whose body is the original body plus the step expression; a while loop becomes a Loop node with an empty initialiser; and redundant nested blocks are flattened. Names are abstracted by the lexer, so a symbol table is not needed for comparison. For the planned Python converter, static types will come from required type hints, since dynamic typing cannot be translated to C in general.

Analysis and scoring. The token-level score is the Jaccard index of two fingerprint sets, $J = |A \cap B| / |A \cup B|$, supplemented by containment $|A \cap B| / |A|$, which shows what fraction of one program appears in another. The logic-level score combines tree alignment, $2M / (|T1| + |T2|)$, where M is the weighted number of aligned nodes and $|T|$ is the tree size, with the fraction of each tree covered by maximal matching subtrees. Subtrees smaller than three nodes are ignored to avoid noise.

Code generation (planned). The converter completes the pipeline: a language front-end parses Python or Java into a source tree, the IR translator maps it onto the shared Node IR, and the generator prints C. Unsupported constructs will produce a diagnostic with the line number rather than incorrect code.

CHAPTER 4

IMPLEMENTATION DETAILS

4.1 Screenshots

This section gives the evidence collected from the prototype. Console output from the early prototype modules is shown first, followed by screenshots of the web application.

Test programs. The comparisons use an original factorial function, a copy in which variables were renamed and comments added, a restructured version in which the for loop is written as a while loop, and an unrelated Fibonacci function.

```
/* original */
int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result = result * i;
    }
    return result;
}
/* restructured: same logic, for loop written as while, names changed */
int fact(int num) {
    int i = 1;
    int total = 1;
    while (i <= num) {
        total = total * i;
        i++;
    }
    return total;
}
```

Figure 4.1: *Programs used in the evaluation*

Lexer output. For the first function of a sample program the normalised token stream was:

```
int ID ( int ID , int ID ) { int ID = ID + ID ; return ID ; }
```

Figure 4.2: *Normalised token stream produced by the lexer*

Parser output. The parser produced the following AST for the factorial function. Names and literals appear as ID and NUM, and the for loop is a ForStmt node with initialiser, condition, step and body.

```
Program
  FunctionDecl(int)
    Param(int)
      ParamName(ID)
    Block
      DeclStmt(int)
        Declarator(ID)
          Init
            Literal(NUM)
      ForStmt
        DeclStmt(int)
          Declarator(ID) > Init > Literal(NUM)
        BinaryOp(<=)
          Identifier(ID)
          Identifier(ID)
        PostfixOp(++)
          Identifier(ID)
        Block
          ExprStmt
            Assign(=)
              Identifier(ID)
              BinaryOp(*)
                Identifier(ID)
                Identifier(ID)
      ReturnStmt
        Identifier(ID)
```

Figure 4.3: *AST of the factorial function (Init chain abbreviated on one line)*

Token-level similarity. Table 4.1 gives the winnowing results with k = 5 and w = 4.

Table 4.1: Token-level similarity

Pair compared	Run	Jaccard	Containment
Original vs renamed and commented copy	1	1.0000	1.0000
Original vs unrelated (Fibonacci)	1	0.3103	0.6429
Original vs unrelated (Fibonacci)	2	0.1935	not recorded
Original vs for-to-while restructured copy	2	0.2727	not recorded

The renamed copy is detected perfectly, as expected from token normalisation. The restructured copy scores low because the token sequence changes when a for loop becomes a while loop, which shows the limit of token-level analysis. The same unrelated pair scored differently in the two runs (0.3103 and 0.1935); the cause is discussed in Section 4.2.

Logic-level similarity before canonicalisation. Table 4.2 shows the first AST comparison, made before the canonicalisation pass existed.

Table 4.2: AST similarity without canonicalisation

Pair compared	Tree alignment	Overall logic similarity
Original vs for-to-while restructured copy (same logic)	0.3860	0.5967
Original vs Fibonacci (different logic)	0.6849	0.6849

The ranking is the wrong way round: the same logic scores lower than different logic, because ForStmt and WhileStmt are different node kinds and the mismatch propagates down the tree. This result is the reason

for the canonicalisation pass, which has been written but whose effect on these scores has not yet been measured.

Web application. The CodeGuard web interface has four screens: Editor & Sandbox, Compiler Pipeline, Plagiarism Report and Submissions. Figure 4.4 shows the Editor & Sandbox screen. The user chooses a language (C subset, C++, Python or Java), loads a template program, supplies standard input, and uses the Run, Check Plagiarism, Pipeline View and Store controls. The code is edited in the Monaco editor, and program output appears in the Execution Output panel.

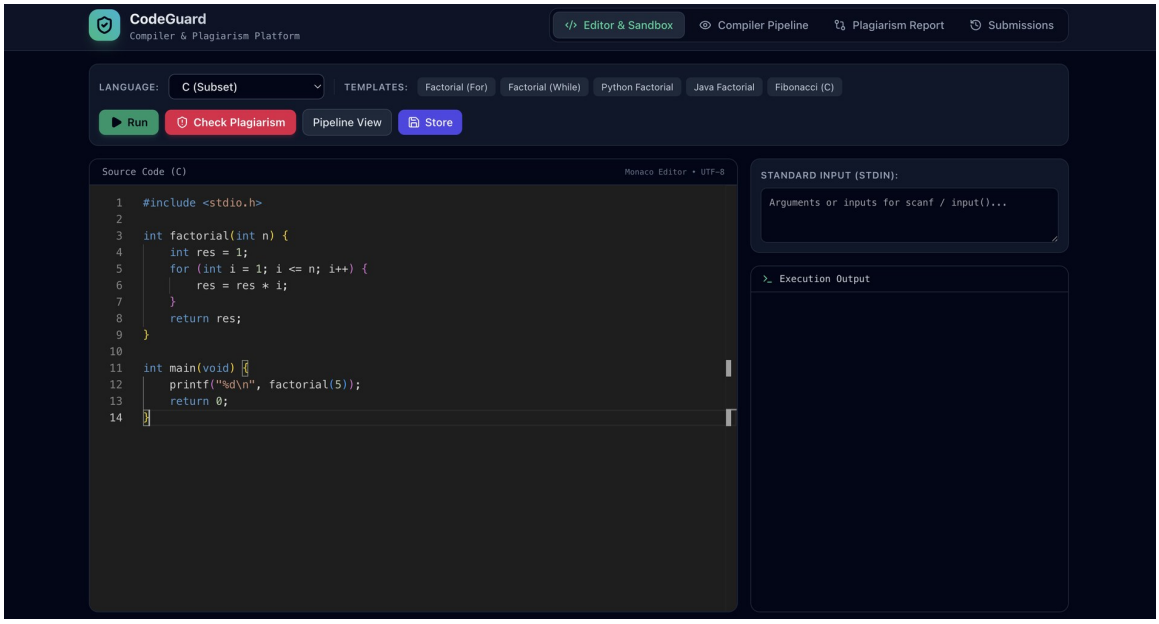


Figure 4.4: *Editor and sandbox screen*

Figure 4.5 shows the plagiarism report for the C factorial program (query A) checked against the stored submissions. The closest match is HW1_Factorial_Charlie.py, a Python implementation of the same algorithm. The token level (winnowing, k = 5, w = 4) gives 54%, the logic level (canonical AST alignment) gives 95%, five maximal matching subtrees are found, and the overall weighted score of 79% is labelled likely copied. The gap between the two scores is the expected behaviour: the loop syntax of C and Python

differs, so tokens match less, while the canonical structure is almost identical. Matched regions are highlighted in both programs, and the other stored submissions are ranked by score above the code panels.

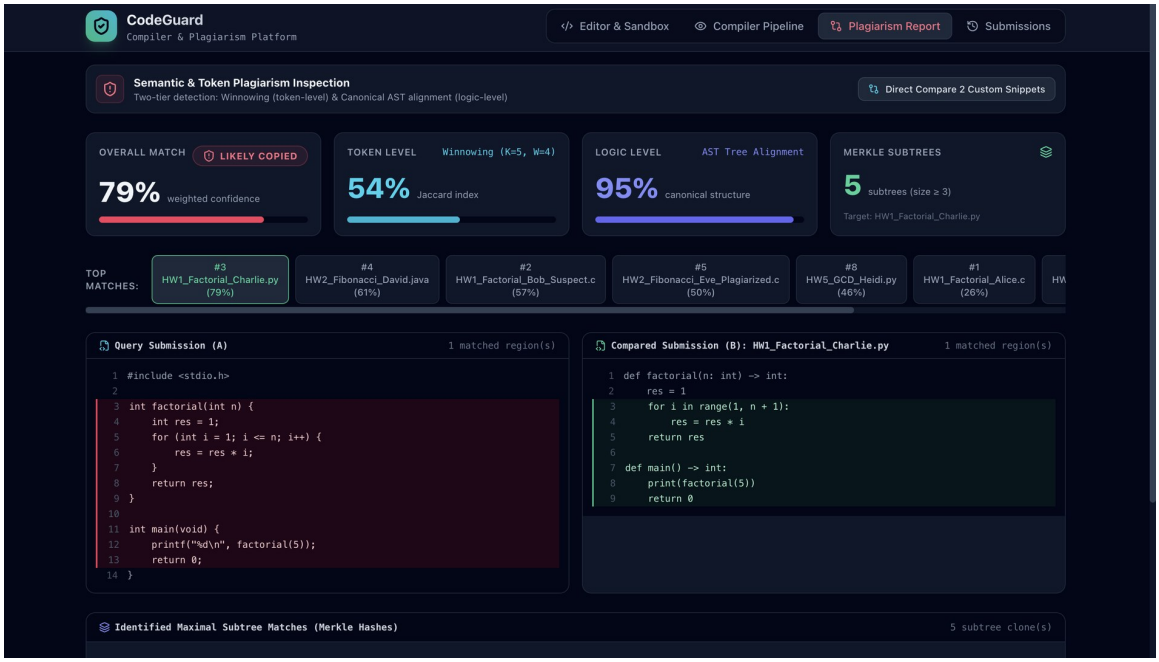


Figure 4.5: *Plagiarism report: token level, logic level, Merkle subtrees and highlighted matches*

The Direct Compare panel (Figure 4.6) compares any two pasted snippets without storing them. Here Code A is the C for-loop factorial and Code B is a Python factorial written with a while loop and different variable names, which tests cross-language and loop-rewrite detection together.

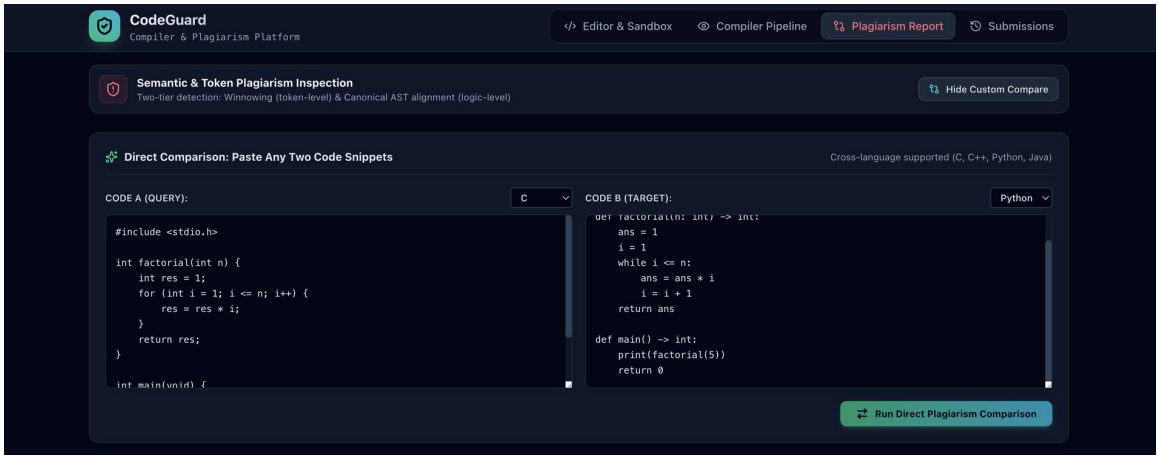


Figure 4.6: *Direct comparison of a C and a Python snippet*

The Submissions screen (Figure 4.7) lists the indexed submissions, here eight programs in C, Python and Java covering factorial, Fibonacci, bubble sort, binary search and GCD. Any two entries can be selected and compared.

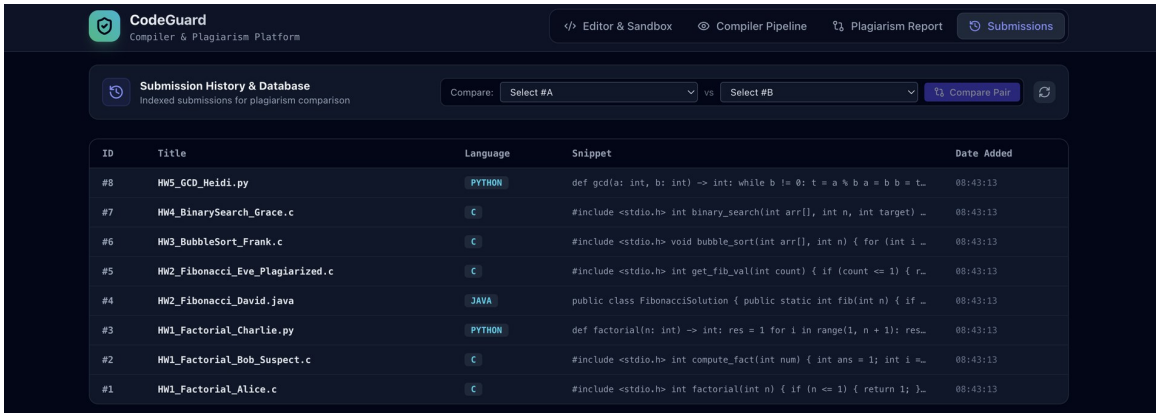


Figure 4.7: *Submission history and database*

Compilation and error detection. The lexer reports an unterminated block comment with its line and column, and the parser reports an unexpected token with its line, then recovers at the next statement boundary. The sandbox is intended to report gcc diagnostics for compilation errors. Screenshots of a successful run and of an error case are still to be added.

[Insert screenshot: Run output in the Execution Output panel for a successfully compiled program]

Figure 4.8: *Successful compilation and execution (to be inserted)*

[Insert screenshot: compile error or syntax-error diagnostic shown for an invalid program]

Figure 4.9: *Error detection (to be inserted)*

4.2 Current Progress and Challenges

Table 4.3: Module status

Module	Status	Remarks
Lexer	Completed	Tested on sample programs
Winnowing engine	Completed	Hash function must be made deterministic (see challenges)
Parser and AST (C subset)	Completed	do-while, switch, ternary and casts not yet supported
AST canonicaliser	Implemented	Effect on similarity scores still to be measured
AST similarity engine	Completed	Size guard for large trees pending
Python and Java front-ends	Completed	Cross-language comparison shown in Figures 4.5 and 4.6
C code generator	Pending	To be verified; output screenshots not yet collected
Execution sandbox	In progress	Run control present in the interface; execution evidence still to be added
Web interface and submission database	Completed	Four screens and eight stored submissions (Figures 4.4 to 4.7)

Challenges encountered

- **Grammar size.** C++ has one of the hardest grammars to parse by hand. A parser generator or libclang would save effort, but libclang could not be installed in the development environment and hand-written phases are the focus of the course. The scope was therefore limited to a C subset with graceful skipping of unsupported constructs.
- **Structure versus names.** The first AST comparison ranked restructured code below unrelated code (Table 4.2). The fix is the canonicalisation pass; its verification on the test corpus is the next step.
- **Non-deterministic fingerprints.** The prototype hashed tokens with Python's built-in hash(), whose string values change between runs, so the fingerprints and the Fibonacci score differed between runs (0.3103 and 0.1935). A fixed hash function such as FNV-1a will replace it.
- **Dynamic typing.** Python cannot be translated to C without static types. The converter will require type hints and will refuse constructs it cannot translate, with a clear message.
- **Score calibration.** In Figure 4.5, programs that implement different algorithms (Fibonacci and GCD) still receive overall scores of 46% to 61% against the factorial query. The thresholds for suspicious and likely copied must be calibrated on a labelled corpus before a score is used as evidence.
- **Cost of tree alignment.** The dynamic-programming alignment grows quickly with tree size. Large trees will use subtree hashing only, and an inverted index will shortlist candidates.

4.3 Conclusion and Future Enhancements

Conclusion. The project shows that the front-end phases of a compiler, namely lexical analysis, parsing and tree normalisation, can be reused to measure similarity between programs. The token layer detects renamed and reformatted copies reliably in the prototype tests, and the canonicalisation pass addresses a real weakness of naive AST comparison. The web application brings the lexers, parsers, similarity engines and a submission database together, and it compares a C submission with a Python submission of the same

algorithm (token 54%, logic 95%). Verification of the sandbox and of the C code generator, and calibration of the scoring thresholds, remain to be done.

Future enhancements

- Complete and verify the sandbox and the C code generator, and check that converted programs produce the same output as the originals.
- Measure the canonicalisation pass on a corpus of several problem families, each with original, renamed, restructured and different-algorithm variants, and calibrate the similarity thresholds on it.
- Add further equivalences: ternary versus if/else, commutative operand ordering, compound assignment versus expanded assignment, and De Morgan rewrites of conditions.
- Add control-flow-graph comparison for resistance against statement reordering.
- Extend the Submissions screen with a class-wide similarity matrix and export of reports.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, "Compilers: Principles, Techniques, and Tools," 2nd ed., Pearson Education, 2006.

[2] S. Schleimer, D. S. Wilkerson and A. Aiken, "Winnowing: Local Algorithms for Document Fingerprinting," in Proc. ACM SIGMOD International Conference on Management of Data, 2003, pp. 76-85.

[3] L. Prechelt, G. Malpohl and M. Philippsen, "Finding Plagiarisms among a Set of Programs with JPlag," Journal of Universal Computer Science, vol. 8, no. 11, pp. 1016-1038, 2002.

[4] L. Jiang, G. Misherghi, Z. Su and S. Glondu, "DECKARD: Scalable and Accurate Tree-Based Detection of Code Clones," in Proc. 29th International Conference on Software Engineering (ICSE), 2007, pp. 96-105.

[5] K. Zhang and D. Shasha, "Simple Fast Algorithms for the Editing Distance between Trees and Related Problems," SIAM Journal on Computing, vol. 18, no. 6, pp. 1245-1262, 1989.

[6] T. Kamiya, S. Kusumoto and K. Inoue, "CCFinder: A Multilinguistic Token-Based Code Clone Detection System for Large Scale Source Code," IEEE Transactions on Software Engineering, vol. 28, no. 7, pp. 654-670, 2002.

[7] B. Roziere, M.-A. Lachaux, L. Chanussot and G. Lample, "Unsupervised Translation of Programming Languages," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[8] F. Iffath et al., "Online Judging Platform Utilizing Dynamic Plagiarism Detection Facilities," Computers, MDPI, 2021.

[9] "Software tool for detecting plagiarism in computer source code," US Patent 7,503,035 B2.

[10] "Source code similarity detection using digital fingerprints," US Patent 11,099,842.

[11] "Multi language software code analysis," US Patent 8,869,120.

[12] "Methods and arrangements for unified program analysis," US Patent 8,640,107.